ГБПОУ «Межрегиональный институт дизайна и сервиса» (МИДиС)
Специальность 09.02.07 — Информационные системы и программирование
Дипломный проект · 4 курс · 2025–2026 уч. г.

ТЕХНИЧЕСКОЕ ЗАДАНИЕ

на разработку веб-сервиса

AI Traveler

Интеллектуальный веб-планировщик путешествий

Разработчик: Садриев Эдуард Романович
Научный руководитель: _____
Дата составления: Апрель 2026
Версия документа: **1.0**

Челябинск, 2026

1. Общие сведения

В данном разделе приводится идентификационная информация о проекте, его участниках и плановых сроках разработки.

Наименование системы	AI Traveler
Тип разработки	Дипломная выпускная квалификационная работа (ВКР)
Организация	ГБПОУ МИДиС (СПО), г. Челябинск
Специальность	09.02.07 — Информационные системы и программирование
Разработчик	Садриев Эдуард Романович, студент 4 курса
Тип продукта	Веб-сервис (SPA + REST API)
Целевая аудитория	Физические лица — самостоятельные путешественники
Язык интерфейса	Русский (с заделом на мультиязычность)
Дата составления ТЗ	Апрель 2026
Плановый срок защиты	Июнь 2026
Версия документа	1.0

2. Назначение и цели создания системы

2.1 Назначение

AI Traveler — веб-сервис для планирования путешествий на базе искусственного интеллекта. Система позволяет пользователю в несколько шагов сформировать персонализированный маршрут: задать направление и бюджет в свободной форме, получить интерактивное визуальное дерево вариантов (жильё, транспорт, события), выбрать предпочтительные опции, просмотреть маршрут на карте и экспортировать итоговый план в PDF.

2.2 Проблема, которую решает система

Самостоятельное планирование поездки требует обхода множества разрозненных сервисов: поиск авиабилетов, выбор отеля, составление программы, подсчёт бюджета. Этот процесс занимает от нескольких часов до нескольких дней. AI Traveler агрегирует весь процесс в единый диалоговый интерфейс и сокращает время планирования до нескольких минут.

2.3 Цели разработки

- Реализовать диалоговый AI-планировщик в форме трёхшагового мастера.
- Обеспечить визуализацию маршрута в виде интерактивного дерева вариантов.
- Отобразить точки маршрута на интерактивной карте (Mapbox GL JS).
- Предоставить динамический расчёт суммарного бюджета поездки.
- Реализовать хранение истории поездок и раздел «Избранное».
- Обеспечить экспорт маршрута в PDF-документ.

– Заложить модульную архитектуру для последующего масштабирования и интеграции реального LLM API.

3. Пользовательский сценарий (UX Flow)

Интерфейс построен как трёхшаговый мастер планирования. Ниже описана основная пользовательская цепочка на основе согласованных макетов.

Шаг 0. Авторизация / Регистрация

– Если пользователь не авторизован — показывается экран входа.
– Поля входа: Логин, Пароль; чекбокс «Запомнить меня»; кнопка «Войти» (акцент — оранжевый).
– Ссылка «Ещё не зарегистрированы?» ведёт на экран регистрации.
– Регистрация: Почта, Номер телефона, Фамилия, Имя, Логин, Пароль, Повторите пароль.
– Footer содержит сводную статистику платформы: Users / Trips / Avg $.

Шаг 1. Выбор направления

– Главный экран: заголовок «Куда отправимся?» и карточка направления.
– Текстовое поле со свободным вводом (placeholder: «Предложи поездку на Пхукет с бюджетом в $1200»).
– Динамический бейдж «Приблизительная сумма: $X» обновляется по мере ввода.

Шаг 2. Уточнение параметров

– Фильтр-панель (выделена оранжевой рамкой): «Когда» / «Обратно» / «N пассажиров» / «Класс».
– Дерево первого уровня: Направление → Место проживания + Рейсы.
– Сумма пересчитывается с учётом выбранных параметров.

Шаг 3. Детализация маршрута

– Дерево расширяется: к «Место проживания» добавляются отель + события; к «Рейсам» — конкретные рейсы.
– Итоговая сумма пересчитывается автоматически.
– Доступные действия: сохранить в историю, добавить в избранное, экспортировать PDF.

Бургер-меню (глобальная навигация)

– Открывается боковая панель: Профиль, Избранное, История поездок, Настройки, Выйти.

История поездок

Список карточек: фото, направление + отель, итоговая стоимость (выделена оранжевым). Действия: открыть, редактировать, экспорт PDF.

Избранное

Grid-галерея (3 колонки) из карточек-мест с изображением и подписью.

Мобильная версия

Дерево маршрута перестраивается в вертикальный список: Направление → Отель → Событие 1 → Событие 2 → Событие 3 → сумма.

4. Функциональные требования

4.1 Управление пользователями

№	Требование	Описание	Приоритет
F-01	Регистрация	Поля: почта, телефон, ФИО, логин, пароль×2. Валидация, bcrypt.	HIGH
F-02	Авторизация JWT	Вход по логину/паролю. Access + refresh токены. «Запомнить меня».	HIGH
F-03	Профиль	Просмотр и редактирование: имя, фото, контакты.	MEDIUM
F-04	Выход	Инвалидация токена, очистка сессии.	HIGH

4.2 Планирование маршрута

№	Требование	Описание	Приоритет
F-05	Свободный ввод	Текстовое поле. AI парсит: направление, даты, бюджет, предпочтения.	HIGH
F-06	Фильтр-панель	Уточнение: даты туда/обратно, кол-во пассажиров, класс.	HIGH
F-07	Дерево маршрута	AI (моки на MVP) строит дерево: направление → жильё + транспорт + события.	HIGH
F-08	Динамический бюджет	Бейдж «Приблизительная сумма: $X» обновляется при изменении параметров.	HIGH
F-09	Редактирование	Пользователь может менять узлы дерева: заменить отель, рейс, добавить/убрать событие.	MEDIUM

4.3 Подбор компонентов

№	Требование	Описание	Приоритет
F-10	Подбор жилья	Список отелей/апартаментов: цена/ночь, рейтинг, фото. (MVP: моки)	HIGH
F-11	Подбор транспорта	Перелёты / ж/д / авто с ценой и временем. (MVP: моки)	HIGH
F-12	Подбор событий	Достопримечательности и локальные события. (MVP: моки)	MEDIUM
F-13	Бюджетный калькулятор	Сумма = жильё×ночи + транспорт + события. Разбивка по категориям.	HIGH

4.4 Карта и визуализация

№	Требование	Описание	Приоритет
F-14	Маркеры на карте	Точки маршрута (отель, аэропорт, события) отображаются на Mapbox.	HIGH
F-15	Линии маршрута	Соединительные линии между точками на карте.	MEDIUM
F-16	Синхронизация	Выбор узла в дереве → карта центрируется на соответствующей точке.	MEDIUM

4.5 История, избранное, экспорт

№	Требование	Описание	Приоритет
F-17	История поездок	Карточки: направление, отель, итог. Действия: открыть, редактировать, PDF.	HIGH
F-18	Избранное	Grid-галерея мест. Добавление/удаление из избранного.	MEDIUM
F-19	Экспорт PDF	PDF содержит: направление, даты, компоненты маршрута, бюджет.	HIGH

5. Нефункциональные требования

5.1 Производительность

- Время ответа API на стандартный запрос — не более 2 секунд.
- Поддержка не менее 20 одновременных пользователей на MVP.
- Генерация PDF — не более 5 секунд (выполняется в фоне через Celery).

5.2 Безопасность

- Авторизация на основе JWT: access-токен (15 мин) + refresh-токен (30 дней).
- Хэширование паролей: bcrypt, cost factor 12.
- Валидация всех входящих данных через Pydantic v2.
- Rate limiting, CORS, HTTPS в продакшн-окружении.

5.3 Совместимость

- Кроссбраузерность: Chrome, Firefox, Safari, Edge (актуальные версии).
- Адаптивный дизайн: Desktop (1280px+) и Mobile (375px+) согласно согласованным макетам.

5.4 Развёртывание

- Полный запуск окружения: docker-compose up — обязательное требование для MVP.
- Изоляция каждого сервиса в отдельном Docker-контейнере.
- Модульная архитектура: модули изолированы и могут масштабироваться независимо.

6. Архитектура системы

6.1 Принцип построения

Система реализована как модульный монолит — все модули изолированы внутри единого бэкенд-приложения с чёткими границами ответственности. Преимущества такого подхода:

- Гибкость и расширяемость: новые модули добавляются без изменения существующих.
- Масштабируемость: при необходимости модули могут быть вынесены в микросервисы.
- Отказоустойчивость: сбой в одном модуле не блокирует работу остальных.
- Асинхронная обработка: тяжёлые операции (PDF, AI-запросы) выполняются через Celery.

6.2 Схема взаимодействия компонентов

Browser (React SPA) → FastAPI (REST API) → PostgreSQL / Redis

FastAPI → LangChain AI Module → [Mock / Real LLM API]

FastAPI → Celery Worker → WeasyPrint PDF Generator

6.3 Стек технологий

Уровень	Технология / Библиотека	Назначение
Frontend	React + Vite + TypeScript	SPA-приложение, быстрая сборка

	CSS (App.css, index.css)	Текущая стилизация прототипа интерфейса
	shadcn/ui	Планируется к внедрению после стабилизации UX
	TanStack Query	Планируется; пока используется прямой fetch
	Zustand	Планируется; пока локальный state в React
	react-map-gl + Mapbox GL JS	Карта запланирована на следующий этап
	React Router v7	Планируется; текущий прототип в одном App.tsx
Backend	Python + FastAPI	REST API, бизнес-логика
	SQLAlchemy + Alembic	ORM + миграции базы данных
	LangChain	AI-модуль (моки → LLM API)
	Redis	Инфраструктурно подключен (контейнер), интеграция в бизнес-логику в процессе
	Celery	Запланировано для фоновых задач (PDF/AI)
	python-jose + Passlib/bcrypt	JWT-авторизация, хэширование паролей
	WeasyPrint + Jinja2	Зависимости подключены, экспорт PDF в разработке
БД	PostgreSQL 16	Основная реляционная СУБД
Инфраструктура	Docker + docker-compose	Контейнеризация всего стека

7. Файловая структура проекта

7.1 Корень репозитория

Путь	Назначение
project-root/	Корень монорепозитория
`backend/`	Python FastAPI приложение
`frontend/`	React + Vite приложение
infra/docker-compose.yml	Оркестрация frontend, api, db, redis
backend/.env.example	Шаблон переменных окружения
`README.md`	Инструкция по запуску проекта

7.2 Backend

Путь	Назначение
backend/app/main.py	Точка входа FastAPI, middleware, подключение роутеров
backend/app/core/config.py	Настройки через Pydantic Settings + .env
backend/app/core/database.py	SQLAlchemy engine, сессии БД
backend/app/models/	ORM-модели: User, Trip, TripNode, TripEdge, Favorite, TripHistory, RoutePoint
backend/app/api/router.py	Технические роуты и подключение модульных роутеров
backend/app/api/v1/auth.py	Регистрация, вход, профиль текущего пользователя
backend/app/api/v1/trips.py	Конструктор поездки: параметры, дерево, выбор жилья/номера/транспорта, summary
backend/app/api/v1/favorites.py	Избранное (каркас, требуется реализация бизнес-логики)
backend/app/api/v1/history.py	История поездок (каркас, требуется реализация бизнес-логики)
backend/app/core/security.py	JWT, хэширование и проверка паролей
backend/alembic/	Конфигурация и миграции Alembic
backend/alembic/versions/	Версии миграций схемы БД
backend/Dockerfile	Docker-образ бэкенда
backend/requirements.txt	Python-зависимости проекта

7.3 Frontend

Путь	Назначение
frontend/src/main.tsx	Точка входа React-приложения
frontend/src/App.tsx	Текущий мастер планирования (прототип интеграции с backend API)
frontend/src/App.css	Стили интерфейса прототипа

frontend/src/index.css	Базовые глобальные стили
frontend/Dockerfile	Docker-образ фронтенда (Vite dev server)
frontend/package.json	Скрипты запуска и зависимости фронтенда
frontend/vite.config.ts	Конфигурация Vite

8. Модель данных (сущности БД)

Ниже описаны основные сущности базы данных PostgreSQL и их связи.

Сущность	Ключевые поля	Связи
users	id, email, username, hashed_password	1:N → trips, favorites, trip_history
trips	id, user_id, title, destination, origin, start_date, end_date, passengers, round_trip	N:1 → users; 1:N → trip_nodes, trip_edges, route_points
trip_nodes	id, trip_id, node_type, title, description, estimated_cost, position_x, position_y	N:1 → trips
trip_edges	id, trip_id, source_node_id, target_node_id	N:1 → trips; связывает узлы trip_nodes
route_points	id, trip_id, name, latitude, longitude	N:1 → trips
favorites	id, user_id, place_name	N:1 → users
trip_history	id, user_id, trip_name, total_cost_label	N:1 → users

alembic_version	version_num	Служебная таблица миграций

Примечание: структура дерева маршрута хранится в таблицах trip_nodes и trip_edges, что упрощает выборки, обновления и дальнейшее расширение логики конструктора.

9. Ключевые API-эндпоинты

Метод	Путь	Описание	Auth
GET	/api/v1/health	Проверка доступности сервиса	Нет
POST	/api/v1/auth/register	Регистрация пользователя	Нет
POST	/api/v1/auth/login	Авторизация, выдача JWT access-токена	Нет
GET	/api/v1/auth/me	Профиль текущего пользователя	JWT
GET	/api/v1/trips	Список поездок	Нет (временный режим прототипа)
POST	/api/v1/trips	Создать поездку	Нет (временный режим прототипа)
GET	/api/v1/trips/{id}	Детали поездки	Нет (временный режим прототипа)
PATCH	/api/v1/trips/{id}/params	Обновление параметров поездки	Нет (временный режим прототипа)
GET	/api/v1/trips/destination/search	Поиск направлений	Нет (временный режим прототипа)
POST	/api/v1/trips/{id}/destination/select	Выбор направления	Нет (временный режим прототипа)

GET	/api/v1/trips/{id}/tree	Получение дерева поездки	Нет (временный режим прототипа)
POST	/api/v1/trips/{id}/accommodations/search	Поиск проживания с фильтрами	Нет (временный режим прототипа)
POST	/api/v1/trips/{id}/accommodations/rooms	Список доступных номеров по параметрам	Нет (временный режим прототипа)
POST	/api/v1/trips/{id}/accommodations/select-room	Выбор номера проживания	Нет (временный режим прототипа)
POST	/api/v1/trips/{id}/transport/search	Поиск транспорта (рейсы туда/обратно)	Нет (временный режим прототипа)
POST	/api/v1/trips/{id}/transport/select	Выбор маршрута транспорта	Нет (временный режим прототипа)
GET	/api/v1/trips/{id}/summary	Итоговая сумма и детализация расчёта	Нет (временный режим прототипа)
GET	/api/v1/favorites	Список избранного (каркас)	Нет (временный режим прототипа)
POST	/api/v1/favorites	Добавить в избранное (каркас)	Нет (временный режим прототипа)
DEL	/api/v1/favorites/{id}	Удалить из избранного (каркас)	Нет (временный режим прототипа)
GET	/api/v1/history	История поездок (каркас)	Нет (временный режим прототипа)

10. Этапы реализации

Этап	Срок	Задачи	Итог

1. Скелет	Неделя 1	Структура проекта, docker-compose, конфиги, БД, Alembic, базовый FastAPI, Vite + React, единый запуск frontend+backend	**Запуск**
2. Авторизация	Неделя 2	Регистрация, вход, JWT, bcrypt, endpoint /auth/me, базовый прототип входа на фронте	**Вход**
3. Планировщик	Неделя 3–4	AI-моки, 3-шаговый мастер, дерево маршрута, фильтр-панель, динамический бюджет	**Маршрут**
4. Карта	Неделя 5	Mapbox через react-map-gl, маркеры, линии, синхронизация с деревом	**Карта**
5. История	Неделя 6	CRUD поездок, карточки истории, grid избранного, бургер-меню, навигация	**Хранение**
6. PDF	Неделя 7	WeasyPrint + Jinja2 шаблон, Celery-задача, скачивание из интерфейса	**PDF**
7. Финал	Неделя 8	Полировка UI, адаптив mobile, тестирование, README, подготовка к защите	**MVP**

11. Критерии приёмки (Definition of Done)

1. Все функциональные требования F-01 — F-19 реализованы и протестированы.
2. Трёхшаговый мастер работает от ввода запроса до отображения дерева маршрута.
3. Карта отображает точки и линии маршрута, синхронизирована с деревом.
4. История поездок и избранное корректно хранятся и отображаются.
5. PDF-экспорт формируется без ошибок и содержит все компоненты маршрута.
6. Проект запускается одной командой docker compose up --build -d из папки infra.
7. Интерфейс корректно отображается на Desktop (1280px+) и Mobile (375px+).
8. Авторизация: регистрация, вход, выход работают корректно, JWT-токены валидируются.

12. Риски и ограничения

Риск	Описание	Митигация
Лимиты бесплатного тира Mapbox	50 000 загрузок карты/мес на бесплатном плане	Использовать MapLibre GL JS как fallback
Переход с моков на реальный LLM API	Возможны изменения в структуре ответа модели	LangChain абстрагирует API — смена в одном файле
Время ответа LLM (10–30 сек)	Долгое ожидание ухудшает UX	Celery + streaming, UX-индикатор загрузки

Сложность визуализации дерева	Нестандартный компонент, нет готового в shadcn	Реализовать через SVG + React или d3-hierarchy

13. Дальнейшее развитие (вне MVP)

- Подключение реального LLM API (Claude / OpenAI) через LangChain.
- Интеграция с реальными API бронирования (Aviasales, Booking, Ostrovok).
- Анимации карты: Motion + GSAP для UI, deck.gl для WebGL-эффектов.
- Интеграция в суперапп MAX (API-first архитектура уже заложена).
- Расширение на СНГ и глобальный рынок: мультиязычность, Mapbox глобальные тайлы.
- Мобильное приложение на React Native с использованием существующего API.
- Совместное планирование: многопользовательские поездки.

Документ составлен: апрель 2026 г. · Версия 1.0

Садриев Эдуард Романович · ГБПОУ МИДиС · Специальность 09.02.07